explain the differences in `subscribeMany` and `querySync`

```
import { AbstractRelay as AbstractRelay,
SubscriptionParams, Subscription } from './abstract-
relay.ts'
import { normalizeURL } from './utils.ts'

import type { Event, Nostr } from './core.ts'
import { type Filter } from './filter.ts'
import { alwaysTrue } from './helpers.ts'

export type SubCloser = { close: () => void }

export type SubscribeManyParams =
Omit<SubscriptionParams, 'onclose' | 'id'> & {
  maxWait?: number
  onclose?: (reasons: string[]) => void
  id?: string
}

export class AbstractSimplePool {
  private relays = new Map<string, AbstractRelay>()
  public seenOn: Map<string, Set<AbstractRelay>> = new
Map()
  public trackRelays: boolean = false

  public verifyEvent: Nostr['verifyEvent']
  public trustedRelayURLs: Set<string> = new Set()

  constructor(opts: { verifyEvent: Nostr['verifyEvent'] }) {
    this.verifyEvent = opts.verifyEvent
  }

  async ensureRelay(url: string, params?: {
connectionTimeout?: number }): Promise<AbstractRelay>
{
```

```
  url = normalizeURL(url)

  let relay = this.relays.get(url)
  if (!relay) {
    relay = new AbstractRelay(url, {
      verifyEvent: this.trustedRelayURLs.has(url) ?
alwaysTrue : this.verifyEvent,
    })
    if (params?.connectionTimeout)
relay.connectionTimeout = params.connectionTimeout
    this.relays.set(url, relay)
  }
  await relay.connect()

  return relay
 }

 close(relays: string[]) {
  relays.map(normalizeURL).forEach(url => {
   this.relays.get(url)?.close()
  })
 }

 subscribeMany(relays: string[], filters: Filter[], params:
SubscribeManyParams): SubCloser {
  return
this.subscribeManyMap(Object.fromEntries(relays.map(ur
l => [url, filters])), params)
 }

 subscribeManyMap(requests: { [relay: string]: Filter[] },
params: SubscribeManyParams): SubCloser {
  if (this.trackRelays) {
    params.receivedEvent = (relay: AbstractRelay, id:
string) => {
      let set = this.seenOn.get(id)
      if (!set) {
        set = new Set()
        this.seenOn.set(id, set)
      }
```

```
      set.add(relay)
    }
  }

  const _knownIds = new Set<string>()
  const subs: Subscription[] = []
  const relaysLength = Object.keys(requests).length

  // batch all EOSEs into a single
  const eosesReceived: boolean[] = []
  let handleEose = (i: number) => {
    eosesReceived[i] = true
    if (eosesReceived.filter(a => a).length ===
relaysLength) {
      params.oneose?.()
      handleEose = () => {}
    }
  }
  // batch all closes into a single
  const closesReceived: string[] = []
  let handleClose = (i: number, reason: string) => {
    handleEose(i)
    closesReceived[i] = reason
    if (closesReceived.filter(a => a).length ===
relaysLength) {
      params.onclose?.(closesReceived)
      handleClose = () => {}
    }
  }

  const localAlreadyHaveEventHandler = (id: string) => {
    if (params.alreadyHaveEvent?.(id)) {
      return true
    }
    const have = _knownIds.has(id)
    _knownIds.add(id)
    return have
  }

  // open a subscription in all given relays
```

```
  const allOpened = Promise.all(
    Object.entries(requests).map(async (req, i, arr) => {
      if (arr.indexOf(req) !== i) {
        // duplicate
        handleClose(i, 'duplicate url')
        return
      }

      let [url, filters] = req
      url = normalizeURL(url)

      let relay: AbstractRelay
      try {
        relay = await this.ensureRelay(url, {
          connectionTimeout: params.maxWait ?
Math.max(params.maxWait * 0.8, params.maxWait - 1000)
: undefined,
        })
      } catch (err) {
        handleClose(i, (err as any)?.message || String(err))
        return
      }

      let subscription = relay.subscribe(filters, {
        ...params,
        oneose: () => handleEose(i),
        onclose: reason => handleClose(i, reason),
        alreadyHaveEvent: localAlreadyHaveEventHandler,
        eoseTimeout: params.maxWait,
      })

      subs.push(subscription)
    }),
  )

  return {
    async close() {
      await allOpened
      subs.forEach(sub => {
        sub.close()
```

```
      })
    },
  }
}

subscribeManyEose(
  relays: string[],
  filters: Filter[],
  params: Pick<SubscribeManyParams, 'id' | 'onevent' |
'onclose' | 'maxWait'>,
): SubCloser {
  const subcloser = this.subscribeMany(relays, filters, {
    ...params,
    oneose() {
      subcloser.close()
    },
  })
  return subcloser
}

async querySync(
  relays: string[],
  filter: Filter,
  params?: Pick<SubscribeManyParams, 'id' | 'maxWait'>,
): Promise<Event[]> {
  return new Promise(async resolve => {
    const events: Event[] = []
    this.subscribeManyEose(relays, [filter], {
      ...params,
      onevent(event: Event) {
        events.push(event)
      },
      onclose(_: string[]) {
        resolve(events)
      },
    })
  })
}

async get(
```

```
    relays: string[],
    filter: Filter,
    params?: Pick<SubscribeManyParams, 'id' | 'maxWait'>,
  ): Promise<Event | null> {
    filter.limit = 1
    const events = await this.querySync(relays, filter,
params)
    events.sort((a, b) => b.created_at - a.created_at)
    return events[0] || null
  }

  publish(relays: string[], event: Event): Promise<string>[] {
    return relays.map(normalizeURL).map(async (url, i, arr)
=> {
      if (arr.indexOf(url) !== i) {
        // duplicate
        return Promise.reject('duplicate url')
      }

      let r = await this.ensureRelay(url)
      return r.publish(event)
    })
  }
}
```

The `subscribeMany` and `querySync` methods in the provided code serve different purposes in managing subscriptions and querying events from multiple relays.

1. **subscribeMany**:
   - This method is responsible for subscribing to events from multiple relays simultaneously.
   - It takes an array of relay URLs, associated filters for each relay, and subscription parameters.
   - It establishes subscriptions with each relay and manages the received events and closure notifications.
   - The method returns a `SubCloser` object, which allows closing all subscriptions opened by this method.
2. **querySync**:

- This method is designed for synchronous querying of events from multiple relays.
- It takes an array of relay URLs, a filter for event selection, and optional subscription parameters.
- Internally, it utilizes `subscribeManyEose` to open subscriptions, but waits for the subscriptions to close before resolving with the collected events.
- It returns a Promise that resolves with an array of events retrieved from the relays.

Key Differences:

- **Purpose**:
  - `subscribeMany` is for continuous event subscriptions.
  - `querySync` is for synchronous querying of events, effectively acting as a one-time subscription and retrieval.
- **Usage**:
  - `subscribeMany` is useful when you need continuous updates on events matching certain criteria from multiple relays.
  - `querySync` is handy for scenarios where you need to fetch events at a particular point in time from multiple relays without maintaining a continuous subscription.
- **Return Values**:
  - `subscribeMany` returns a `SubCloser` object for managing the opened subscriptions.
  - `querySync` returns a Promise that resolves with an array of events retrieved from the relays.

In summary, `subscribeMany` is for ongoing subscriptions, while `querySync` is for fetching events synchronously. Both methods facilitate interaction with multiple relays but cater to different use cases and retrieval mechanisms.

what about pool.get()?

The `get` method in the provided code serves as a convenience function to retrieve a single event that matches the specified filter from multiple relays. Here's an explanation

of its functionality and how it differs from `subscribeMany` and `querySync`:

1. **get**:
   - This method is designed to fetch a single event that matches the provided filter from multiple relays.
   - It takes an array of relay URLs, a filter for event selection, and optional subscription parameters.
   - Internally, it utilizes `querySync` with the provided filter and parameters, but ensures that only one event is fetched by setting `filter.limit = 1`.
   - After fetching events, it sorts them by their creation time and returns the latest event, or `null` if no event is found.
   - It offers a simplified interface for retrieving a single event without the need for managing subscriptions explicitly.

Key Differences:

- **Purpose**:
  - `get` is specifically for retrieving a single event matching the provided filter from multiple relays.
  - `subscribeMany` is for continuous event subscriptions.
  - `querySync` is for synchronous querying of events, potentially retrieving multiple events.
- **Usage**:
  - `get` is suitable when you only need the latest event that matches the filter from multiple relays.
  - `subscribeMany` is appropriate for continuous updates on events matching certain criteria.
  - `querySync` is useful for fetching events at a specific point in time, potentially retrieving multiple events.
- **Return Value**:
  - `get` returns either the latest event that matches the filter or `null`.
  - `subscribeMany` returns a `SubCloser` object for managing opened subscriptions.
  - `querySync` returns a Promise that resolves with an array of events retrieved from the relays.

In summary, `get` provides a simplified way to fetch a single event from multiple relays, whereas `subscribeMany` and `querySync` are more versatile methods for managing